BLACKJACK GAME APPLICATION

Project Report

Course Name: *CPR 210 Mobile Application Development*

Project Title: *Blackjack Game Application*

Student Name: *Kenan Esat Şen*

Student Number: *202246031*

Instructor: *Lect. Efe Çiftci*

University: *Çankaya University*

Department: *Computer Programming*

Date: *31.12.2025*

TABLE OF CONTENTS

1. Introduction
2. Technologies Used
3. User Interface Development with Jetpack Compose
4. Game Data Structures and Logic
    - 4.1 Suit Enumeration
    - 4.2 Card Data Class
    - 4.3 Deck Class
    - 4.4 Hand Value Calculation
    - 4.5 Blackjack Detection
5. Game State Management
6. Game Flow and Logic Implementation
7. User Interface Features
8. Conclusion

1. Introduction

The aim of this project is to develop a **Blackjack (21) card game application** for Android devices. The application is implemented using **Kotlin** and follows the **standard rules of Blackjack.** The player can interact with the game by choosing actions such as ***Deal*, *Hit*,** and ***Stand*,** while the dealer plays automatically according to predefined rules. The project focuses on applying both game logic and modern Android user interface development techniques.

2. Technologies Used

The following technologies and tools were used in the development of this project:

- **Kotlin** programming language
- **Android Studio**
- **Jetpack Compose** (Material 3)
- **State management** using
- (**remember**) and **(mutableStateOf)**
- Random number generation using (**kotlin.random.Random**)

3. User Interface Development with Jetpack Compose

Jetpack Compose is used as the main framework for building the user interface of the application. Unlike traditional Android development, which relies on XML layout files, Jetpack Compose allows the entire interface to be written directly in Kotlin using **composable functions**.

Jetpack Compose follows a **declarative programming model**, meaning the UI is defined based on the current state of the application. When the state changes, the UI automatically updates without requiring manual intervention.

In this project, game-related states such as the player's cards, dealer's cards, and game status are stored using **remember** and **mutableStateOf**. Any change in these values automatically triggers **UI recomposition**, ensuring that the interface always reflects the current game state.

4. Game Data Structures and Logic

4.1 Suit Enumeration

The Suit enumeration represents the four types of playing card suits: **Spades, Hearts, Diamonds, and Clubs.** Each suit also contains a visual symbol that is displayed on the screen.

4.2 Card Data Class

The Card data class represents a single playing card. It contains information about the card's rank and suit. The card is displayed in a readable format such as ***A♠* or *10♦*.**

4.3 Deck Class

The **Deck class** is responsible for creating a standard **52-card deck**. The cards are shuffled randomly at the beginning of the game. Cards are drawn from the deck during gameplay, and if the deck becomes empty, a new deck is created automatically to maintain continuity.

4.4 Hand Value Calculation

The hand value is calculated based on standard Blackjack rules:

- Number cards have their face value.
- Face cards (**Jack, Queen, King**) are worth **10 points.**
- **Aces** are initially valued at **11 points** but can be reduced to **1 point** if the total hand value exceeds **21.**

4.5 Blackjack Detection

A Blackjack hand is detected when a hand consists of exactly two cards and the total value equals **21.**

5. Game State Management

The game uses a state-based system to manage gameplay. The states include:

- **READY:** The game is waiting for a new round to start.
- **PLAYER_TURN:** The player can choose to Hit or Stand.
- **FINISHED:** The round is completed and the result is shown.

These states control both game logic and the availability of user interface elements.

6. Game Flow and Logic Implementation

At the start of each round, two cards are dealt to both the player and the dealer. The player may then choose to draw additional cards (**Hit**) or end their turn (**Stand**). Once the player **stands**, the dealer automatically draws cards until reaching a minimum **hand value of 17**. The winner is determined by comparing the final hand values.

7. User Interface Features

- The dealer’s second card and hand value are hidden during the player’s turn.
- Clear game messages inform the player about game progress and results.
- Large, color-coded buttons improve usability.
- A **“New Round”** button allows the game to be **reset** easily.

8. Conclusion

In conclusion, this project successfully demonstrates the development of a ***Blackjack game application*** using **Kotlin** and **Jetpack Compose**. The application combines correct game logic with a modern and reactive user interface. Through the use of **declarative UI principles** and **state management**, the project provides a strong example of contemporary Android application development.